Real-world (config + logging together) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Hint 3 · Hint 4 · Solution

Work through in order — don't jump ahead until you've genuinely tried.

Where this exercise is saved: its own folder, `practice/config_logging_wiring/`, because it needs more than one file:

```
practice/config_logging_wiring/
├── exceptions.py        MissingConfigError
├── config.py            load_config() -> Config
├── logging_setup.py     get_logger(name)
├── main.py              wires the two together
└── .env                 you create this; never committed
```

Run it: `cd practice/config_logging_wiring && python main.py` — from inside the folder, so `from config import load_config` finds the file sitting next to it.

Builds on: `practice/logging_practice.py` (Intermediate part 2) — **copy** its `get_logger()` into `logging_setup.py`, same name, same signature. And `practice/custom_errors_practice.py` (Basic) — the same custom-error pattern, re-written here as `MissingConfigError` in `exceptions.py`. Copies, not imports: this folder runs on its own. The hints below are about `main.py`, the one genuinely new file; the Solution shows all four in full.

- Hint 1
- Hint 2
- Hint 3
- Hint 4

Hint 1 — What's the idea here?

This exercise is about **wiring**, not new logic — you already built `load_config()` and `get_logger()` separately. Now `practice/config_logging_wiring/main.py` just needs to call both, and use them together: get the config, get a logger, then log something that includes a value from config.

Think about the order: which one do you call first, and why does that order matter?

Hint 2 — The exact pieces

`from config import load_config` and `from logging_setup import get_logger` at the top of `practice/config_logging_wiring/main.py`. Call `load_config()` first — if it's going to fail (a missing key), you want that to happen before you've done anything else, including setting up logging.

Use an f-string to build your log message with a value from the config object: `logger.info(f"... {config.some_field}...")`.

To handle a startup failure cleanly, wrap `load_config()` in `try` / `except MissingConfigError as e:`, and on failure `print(f"Startup failed: {e}")` followed by `sys.exit(1)` — a normal Python program exits with code `1` to signal "something went wrong" to whatever ran it.

Hint 3 — The plan, in pseudocode

```
in practice/config_logging_wiring/main.py:
    import load_config from config.py
    import get_logger from logging_setup.py
    import sys and MissingConfigError

    try:
        # do this first — fail fast if something's missing
        config = load_config()
    except MissingConfigError as e:
        print an error message and sys.exit(1)

    logger = get_logger(__name__)

    logger.info a message that includes config.log_level (or any field you have)

run it, and confirm the line appears on screen with the config value inside it
run it again with a broken/missing config value, and confirm you get a clean
one-line error instead of a Python traceback
```

Hint 4 — Almost the whole thing

```
# practice/config_logging_wiring/main.py
import sys

from config import load_config
from exceptions import MissingConfigError
```

```
from logging_setup import get_logger

try:
    config = load_config()
except MissingConfigError as e:
    print(f"Startup failed: {e}")
    sys.exit(1)

logger = get_logger(__name__)

logger.info(f"Loaded config with log level: {config.log_level}")
```

Run it with `cd practice/config_logging_wiring && python main.py` and confirm the line prints. Then break your config on purpose (delete a required value) and confirm you get the clean one-line "Startup failed: ..." message instead of a traceback, then compare against the Solution.

Full solution: Show me the solution